

Implement Performance Measurement Experiment

Tanvir Ahmed (1386435)
tanvir.ahmed@stud.fra-uas.de

Abstract— In order to optimize computational resources, machine learning applications require efficient performance assessment. This project's aim is to implement a performance measurement experiment focused on MultiSequence Learning using Hierarchical Temporal Memory, which is a version of biologically inspired machine learning framework that processes with sequential data over Sparse Distributed Representations (SDRs). To improve computational efficiency, SDRs assist effective encoding of the input sequences. As a means to evaluate the performance, the console application allows users to specify parameters such as sequence folder, output file location, number of CPU cores, CPU affinity, which automatically runs experiment using interface IExperiment, primarily focused on MultiSequence Learning experiment and records key performance metrics in a CSV in the result folder and performance diagrams are generated with the metrics to analyze the relationship between the hardware configurations software environments, and computational efficiency.

Keywords— Performance Measurement, MultiSequence Learning, Hierarchical Temporal Memory, Learning Time Analysis, Temporal Memory, Machine Learning Benchmarking, Sequence Learning, Computational Efficiency.

1. INTRODUCTION

To ensure optimal resource utilization, especially when processing large datasets, machine learning models need effective performance evaluation. MultiSequence Learning method is designed in a way to detect and predict patterns across multiple sequences which is particularly important for time-series analysis, anomaly detection and forecasting tasks. In this project, we implemented a console-based application that allows users to run the experiment and at the same time can monitor the key performance metrics such as learning time, CPU usage, and memory consumption. This application can manage variable configurable parameters, including the sequence dataset location, number of CPU cores, .NET runtime version giving the outlook of performance analysis.

To implement multi-sequence learning using HTM, the process starts with Sparse Distributed Representations (SDRs), where input is converted to binary with a small percentage of active bits and the encoding maintains

semantic similarity which makes the model robust to noise letting it identify similar inputs efficiently. Then the Spatial Pooler processes the SDRs to produce stable and coherent representations. Multisequence learning using HTM is the most effective method for recognizing and predicting patterns throughout multiple input sequences.

This project is aimed at measuring the performance of an HTM based multisequence learning implementation under various conditions. With this experiment, which is implemented around multisequence learning, to quantify the learning time which is required for the HTM to learn sequences of different lengths, evaluate the CPU core utilization, different runtime environments (.NET versions) on training speed and learning time affect the execution performance.

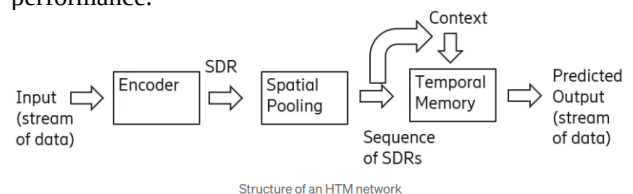


Figure 1: Overview of Sequence Learning [2]

1.1. Background

HTM is aimed at mirroring the brain's hierarchical structure as well as the learning process. HTM encompasses a hierarchically arranged network of nodes, each containing a group of neurotransmitters in the neocortex. Based on their prior experiences, these nodes learn to identify patterns in sensory input and foresee outcomes. The node's models are then adjusted, and predictive accuracy is increased by comparing the predictions with the input data. According to Hawkins, the neocortex creates an ordered set of columns, each of which consists of a collection of neurons that identify patterns in sensory information, in order to educate and make predictions. Higher-level columns represent abstract notions, and these columns interact in a hierarchical framework [3].

1.1.1. Hierarchical Temporal Memory

HTM is aimed at mirroring the brain's hierarchical structure as well as the learning process. HTM encompasses a hierarchically arranged network of nodes, each containing

a group of neurotransmitters in the neocortex. Based on their prior experiences, these nodes learn to identify patterns in sensory input and foresee outcomes. The node's models are then adjusted, and predictive accuracy is increased by comparing the predictions with the input data. According to Hawkins, the neocortex creates an ordered set of columns, each of which consists of a collection of neurons that identify patterns in sensory information, in order to educate and make predictions. Higher-level columns represent abstract notions, and these columns interact in a hierarchical framework [3].

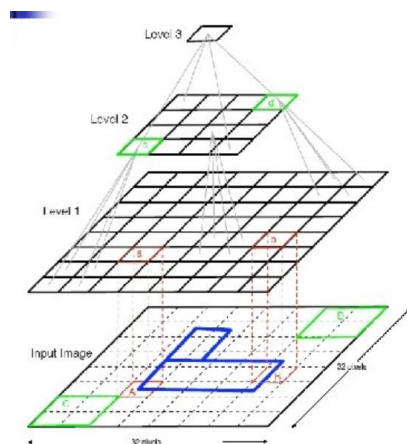


Figure 2: Hierarchical Temporal Memory Structure [4]

1.1.2. Proximal Dendrite Segments

Cells are joined in a column by a proximal dendrite, and synapses are shown by tiny black circles. A legitimate synaptic link with a persistence value higher than the connection threshold is shown by a solid circle. A putative synaptic connection with a persistence value below the connection threshold is represented by an empty circle. After a local inhibitory step, feedback input activates a column if there are sufficient genuine synapses linked to active input bits [14].

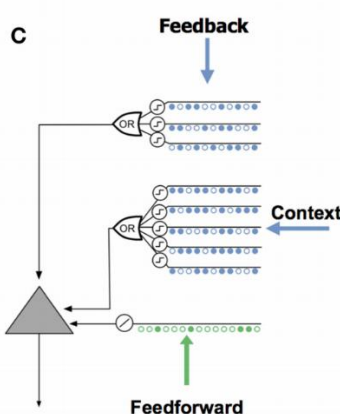


Figure 3: Proximal Dendrite Representation [9]

1.1.3. Distal Dendrite Segments

There are typically around 40 synapses in each of the 130 distal dendritic categories found in each brain cell. It has

one proximal dendritic segment as well. A set of synapses can connect to a subset of other cells within a specific "learning radius," while nearby cells provide lateral input to the distal segments. The dendritic segment remembers the activation status of the surrounding cells by connecting with previously activated cells. The cell enters an anticipatory state if a dendritic segment recognizes the same pattern of cellular activity, that is, if the number of activated synapses on any segment surpasses a certain threshold. Therefore, it is probable that column activation will arise from signalling that feedforward input.

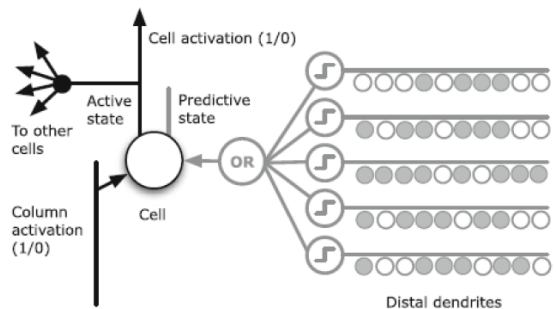


Figure 4: Distal Dendrite Segment at the Cellular Level [13]

1.1.4. Prediction

Its primary function is estimate, which is the basis of intelligence. It makes predictions about the real world using both new incoming data and static representations [15].

1.1.5. Memory

The cortex is not the same as a parallel processor. The brain employs this technique to retrieve output from its enormous memory more rapidly, even if parallel computers do several calculations on input patterns at once to create distinct output patterns. Sequential and regular patterns are spontaneously accumulated and linked in a hierarchical manner by the cortex. Complete patterns may be recovered from incomplete data inputs in both temporal memory and these linked memories.

1.1.6. Connection

In contrast to the conventional ANN neuron model seen in Fig. 5[16], which necessitates a weighted average of inputs followed by a non-linear operation on the total, the HTM neuron model is modelled after cortical neurons. The basis for memory and recall in the brain is provided by biological neurons, which carry out increasingly complex functions and interact through both chemical and electrical impulses, according to recent neuroscience studies.

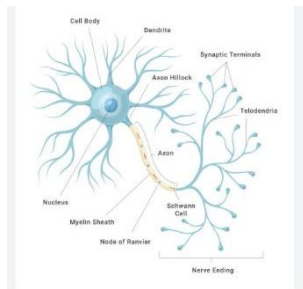


Figure 5: Neuron Structure [11]

Similar mechanisms underlie the communication process: Neuron A becomes electrically charged with the fluid surrounding its membrane whenever it receives an organic signal from another neuron. The electric field travels away from A's soma and down the axon. Vesicles are a group of synaptic storage locations that hold substances produced by the soma. These vesicles fuse with the cell membrane when an electrical charge reaches the point of contact, releasing neurotransmitters into the space between neurons. Neurotransmitters connect to membrane receptors on one of neuron B's dendrites after crossing the synaptic cleft. Before repeating the action, neuron B produces an electrical charge that passes through its axon.

1.1.7. Temporal Memory

A group of active cells that reflect the current context of the incoming data are stored in temporal memory. The active cells alter in response to new input patterns based on how closely the input matches the current context. Furthermore, the software manages a collection of cell connections that symbolize the sequence of patterns previously observed [6]. Temporal Memory may predict the next most probable input pattern based on the present context by using these links. The algorithm fortifies ties between cells that were active during the anticipated sequence if the prediction comes true. The algorithm adjusts the connections to reduce the likelihood that the sequence will recur in the future if the forecast proves to be incorrect.

1.1.8. Encoder

Raw data inputs are received by the encoder, which transforms them into SDRs that highlight the key characteristics of the incoming data. Among the many steps in the encoding process are equalization, noise reduction, and dimensionality reduction. Additionally, over time, the encoder adapts to changes in input concentration and learns to spot patterns in the input data.

In order to identify temporal patterns in the input data, the SP encoder uses a sliding window technique and is designed to handle temporal data. The incoming data is first converted into a continuous stream of binary values, which are subsequently fed into the HTM network as a sequence of SDRs [5].

1.1.9. Spatial Pooler

An unsupervised learning technique called the Spatial Pooler transforms high-dimensional input data into a lower-dimensional SDR that captures the key characteristics of the input data. To do this, it first gives

each input characteristic an arbitrary number of weights, after which it determines how much the input and the weights overlap. After that, the traits that overlap the most are selected and included to the SDR[6].

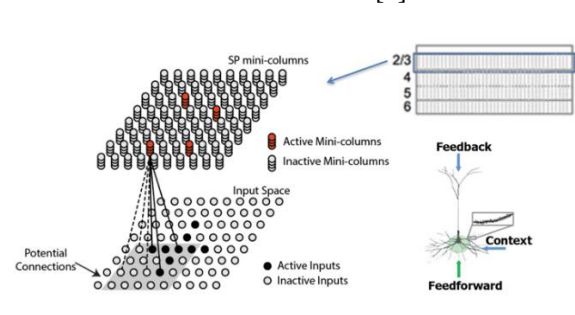


Figure 6: Representation of spatial pooler[12]

1.1.10. Sparse Distributed Representation

SDRs are used in HTM to express patterns of network activity. Every network input is transformed into an SDR, which the network's node hierarchy then uses to predict input in the future.

As a logical means of characterizing sparse, dispersed activation patterns in the neocortex, Hawkins and Ahmad developed SDRs, which are binary sequences with a limited number of active bits (ones) out of a large number of total bits [8].

1.1.11. Multisequence Learning

An HTM-based system called multisequence learning may simultaneously learn and predict many pattern sequences. In multisequence learning, different Temporal Memory modules are used to learn and predict each pattern sequence. The foundation of Ahmad's [7] method is the use of a hierarchical node structure to learn and predict feature sequences at different levels of abstraction. Each Temporal Memory unit essentially assesses and predicts the unprocessed sensory information from a single modality. At higher levels, the nodes comprehend and predict patterns that combine information from several modalities.

2 METHODOLOGY

In order to evaluate multisequence learning performance, developed a console application which uses Neocortex API library, an open-source implementation of HTM algorithms. All the experiments were conducted on a single machine, with controlled variations in CPU core usage and .NET runtime version for performance comparison.

2.1. System Architecture and Components

2.1.1. Input Handler

Implemented in Program.cs, command line arguments are being parses with this component such as --sequence-folder, --out, --experiment, --cores, --affinity. Experiment gets dynamically configured at runtime with this component.

```
InputParameter inputParameter = ParseArguments(args);
ValidateInputDirectory(inputParameter.SequenceFolder);
```

2.1.2 Data Loader

Data Loader uses MultiSequenceInput.cs and Sequence.cs to deserialize the training and testing sequences from JSON files (dataset_*.json, test_*.json) into objects. Data loader do the grouping and pre-processing of the datasets before training.

```
List<Sequence> trainingSequences = LoadDatasets(inputParameter.SequenceFolder, "dataset_*.json");
List<Sequence> testSequences = LoadDatasets(inputParameter.SequenceFolder, "test_*.json");
```

```
public class Sequence
{ public String name { get; set; }
  public double[] data { get; set; } }
```

Source code 1: Data model of Sequence

```
[{"name": "S1",
 "data": [ 0, 2, 5, 6, 7, 8, 10, 11, 13 ]},
 {"name": "S2",
 "data": [ 1, 2, 3, 4, 6, 11, 12, 13, 14 ]},
 {"name": "S3",
 "data": [ 1, 2, 3, 4, 7, 8, 10, 12, 14 ]}]
```

Dataset 1: Sample dataset

2.1.3. Experiment Framework

IExperiment.cs, is the interface that defines a common contract for experiment classes. By using reflection, Program.cs dynamically instantiates a class (i.e. MultiSequenceLearning.cs) implementing this interface. This approach supports future experiments without making any changes in the main program.

```
public class MultiSequenceLearning : IExperiment

private static IExperiment LoadExperiment(string experimentClassName)
{
    Type experimentType = Type.GetType($"{NeocortexApiSamplePerformance.ExperimentClassName}");
    if (experimentType == null || !typeof(IExperiment).IsAssignableFrom(experimentType))
        throw new ArgumentException($"Invalid experiment: {experimentClassName}. Make sure it implements IExperiment.");
    return (IExperiment)Activator.CreateInstance(experimentType);
}

IExperiment experiment = LoadExperiment(inputParameter.ExperimentClass);
```

2.1.4. HTM Engine

This component, which is implemented in MultiSequenceLearning.cs, simulates HTM's Spatial pooler and Temporal Memory mechanisms.

```
public Predictor Run(List<Sequence> sequences)
{
    Console.WriteLine($"Hello NeocortexApi! Experiment {nameof(MultiSequenceLearning)}");
    int inputBits = 100;
    int numColumns = 1024;

    HtmConfig cfg = HelperMethods.FetchHTMConfig(inputBits, numColumns);

    EncoderBase encoder = HelperMethods.GetEncoder(inputBits);

    return RunExperiment(inputBits, cfg, encoder, sequences);
}
```

Here the Run() method performs training, the Predict() function allows sequence prediction. HTM engine processes each sequence as SDR input.

```
var predictor = experiment.Run(processedTrainingData);
var res = predictor.Predict(prev);
```

The table covers Spatial Pooler settings with default values that are often used in HTM research. Selecting the appropriate parameters for the various techniques is crucial in HTM. The performance of HTM is influenced differently by each of these factors. However, we will focus on the influence of certain features, such as the number of active columns per area, global/local inhibition, potential radius, and local area volume.

Using the htm-config class to define a large number of parameters is the first step in using any HTM setup. This phase of the procedure is crucial. All of the HTM parameters that influence picture categorization are included in the table below.

Parameters	Default value
input bits	100
numColumns	1024
CellsPerColumn	25
GlobalInhibition	true
LocalAreaDensity	-1
NumActiveColumnsPerInhArea	0.02* numColumns
PotentialRadius	0.15 * inputBits
MaxBoost	10.0
InhibitionRadius	15
DutyCyclePeriod	25
MinPctOverlapDutyCycles	0.75
MaxSynapsesPerSegment	0.02* numColumns
Activation Threshold	15
Connected Permanence	0.5
Permanence Decrement	0.25
Permanence Increment	0.15
Predicted Segment Decrement	0.1

Table 1: HTM Config parameters

2.1.5. Performance metrics and logging

PerformanceLogger.cs is used to log key metrics such as CPU cores, affinity, RAM usage, CPU speed, and .NET version. These metrics are logged along with training accuracy and learning time for each experiment.

```
PerformanceLogger.LogPerformance(...)
```

2.1.6. Visualization

generate_charts.py, this python script was used to generate performance charts based on the logged CSV data. The pandas library was used for data processing, and for visualization matplotlib and seaborn was used. Learning time, CPU core usage, .NET version and CPU speed were visualized to explore the correlations.

3. IMPLEMENTATION

3.1. Experiment Configuration and Argument Parsing

This experiment is designed in a way that it works it dynamic configuration at runtime through command line arguments which includes the dataset location, number of CPU cores being used and to calculate the number of CPU cores are used decimal affinity values are used.

--sequence-folder <path> --out <output.csv> --cores 1 --affinity 1 --experiment MultiSequenceLearning”

To run the experiment with different .NET runtime version, the runtime versions were manually changed from the projects csproj file with different runtime version.

```
<TargetFramework>net9.0</TargetFramework>
```

Table 2 represents the mapping between decimal CPU affinity values and their corresponding logical processor cores. Every value characterizes a bitmask where set bits specify the cores to which a process or tread can be found.

Decimal Affinity value	Corresponding Cores
1	0
3	0,1
7	0,1,2
15	0,1,2,3
31	0,1,2,3,4
63	0,1,2,3,4,5
127	0,1,2,3,4,5,6
255	0,1,2,3,4,5,6,7

Table 2: CPU Core Binding via Decimal Affinity

Values

InputParameter.cs class was implemented to parse argument from Program.cs.

```
InputParameter inputParameter = ParseArguments(args);
ValidateInputDirectory(inputParameter.SequenceFolder);
```

3.2. CPU Affinity Handling

By using System.Diagnostics.Process, the CPU affinity masking is controlling parallelism:

```
process.ProcessorAffinity = (IntPtr)affinityBitmask;
```

In order to verify the active cores, the method below decodes the bitmask into core indices:

```
private static List<int> GetActiveCores(int affinityBitmask)
```

It confirms consistent benchmarking throughout hardware profiles.

3.3. DataSet Loading

Training and testing sequences are stored as JSON file (dataset_*.json, test_*.json) and loaded using JsonConvert.DeserializeObject:

```
var seqData = ReadDataset(file);

return JsonConvert.DeserializeObject<List<Sequence>>(File.ReadAllText(datasetPath));
```

The sequences are grouped by name and combined using LINQ:

```
var processedTrainingData = trainingSequences
    .GroupBy(seq => seq.name)
    .Select(g => new Sequence { name = g.Key, Data = g.SelectMany(seq => seq.Data).ToArray() })
    .ToList();
```

3.4. Training with Run() via IExperiment

The main experiment logic is extended in the IExperiment interface. The selected experiment class (i.e. MultiSequenceLearning) is dynamically loaded:

```
Type experimentType = Type.GetType($"NeocortexApiSamplePerformance.{experimentClassName}");
IExperiment experiment = (IExperiment)Activator.CreateInstance(experimentType);
```

Model training is initiated with:

```
var predictor = experiment.Run(processedTrainingData);
```

This launches MultiSequenceLearning.cs's Run() implementation which sets up the HTM pipeline:

```
public Predictor Run(List<Sequence> sequences)
{
    int inputBits = 100;
    int numColumns = 1024;
    HtmConfig cfg = HelperMethods.FetchHTMConfig(inputBits, numColumns);
    EncoderBase encoder = HelperMethods.GetEncoder(inputBits);
    return RunExperiment(inputBits, cfg, encoder, sequences);
}
```

3.5. Sequence Prediction and Accuracy Evaluation

Every test sequence is examined component by component. The trained model is utilized to provide predictions:

```
var res = predictor.Predict(prev);
```

Predictions and actual values are compared for each sequence to determine accuracy:

```
double testAccuracy = (double)matchCount / totalPredictions * 100.0;
```

This enables both global and per-sequence assessments of prediction ability.

3.6. Performance Recording and Logging

In order to monitor the performance of the experiment, the application records learning time using Stopwatch, RAM usage via Process.PrivateMemorySize64, CPU Speed using WMI and .NET version using RuntimeInformation.FrameworkDescription. All the performance metrics and accuracy results are logged into an output CSV file using PerformanceLogger.LogPerformance().

3.7. Visualization and Result Interpretation

Implemented a Python script (generate_charts.py) that reads the output CSV file and generated graphs using pandas, matplotlib, and seaborn libraries.

4. RESULT AND DISCUSSION

In this project, the experimental results were gathered from 48 runs of the experiment, 24 time for each .NET 8.014 and .NET 9.0.3. For each runtime, the application was executed with 8 configurations, each using 1 to 8 CPU cores. Every configuration was repeated 3 times to make sure the statistical reliability. The following diagrams are for visual trend analysis from the output metrics that has been logged in output csv (Result.csv).

4.1. Learning Time vs CPU cores

The average learning time with respect to the number of CPU cores used is shown in the bar plot. From the bar plot, a downward trend is displayed as the core count increases

from 1 to 8, which indicates that the model training benefits from parallel computation. However, at 6 cores, an anomaly was observed where the learning time increased abnormally. This sudden spike could be due to temporary system resource contention, garbage collection or other background processes which may affect the performance at the time of that set of trials.

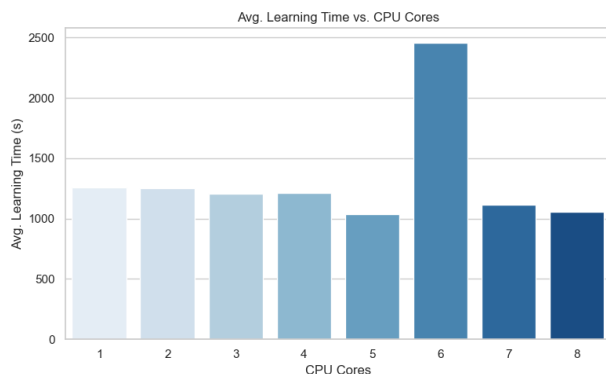


Figure.: Learning time vs CPU Cores

4.2. Learning Time vs CPU Speed

This chart shows the relationship between average learning time and CPU speed, grouped into distinct GHz ranges. All the experiment was run on a machine with a CPU speed range falling between 2.40-2.43 GHz and got this learning time data. Though the limitation of running the experiment on one single machine's CPU frequency doesn't show the deeper trend analysis, this graph shows the consistency of the hardware environment.

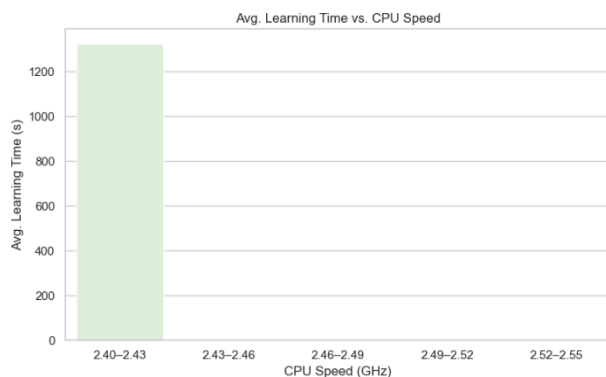


Figure.: Learning time vs CPU Speed

4.3. Learning Time vs .NET Version

This chart compares the average learning time with different .NET versions. .NET runtime version 9.0.3 showed better performance with a lower average learning time (~ 1150s) compared to .NET 8.0.14 (~1390s) average learning time with respect to the number of CPU cores used is shown in the bar plot.

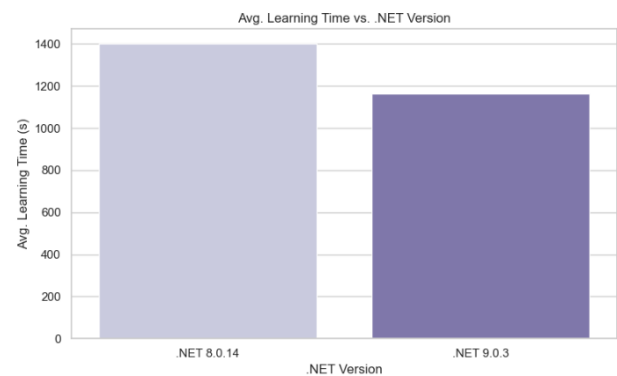


Figure.: Learning time vs .NET version

5. CONCLUSION

In this project a modular and extendable experimental framework for assessing the performance of Hierarchical Temporal Memory (HTM) based learning, especially the MultiSequence Learning model, is successfully implemented. Dynamic control over CPU affinity, core selection, and .NET runtime versions were made possible during execution by including a console-based runner with variable parameters.

Key performance metrics such as learning time, RAM usage, CPU speed, and runtime versions were logged in CSV outputs. The experiments results showed a relationship between CPU core count and learning efficiency, confirming the advantage of parallelization. Also, the comparison among dotnet versions showed significant runtime optimizations, where .NET 9.0.3 outperformed .NET 8.0.14 in most configurations. IExperiment interface which uses reflection makes this application flexible to reuse for wider HTM research and testing.

6. REFERENCES

- [1] "NeoCortexApi". <https://github.com/ddobric/neocortexapi>.
- [2] "Joy bose" Hierarchical Temporal Memory: more biologically plausible than deep learning | by Joy Bose | Analytics Vidhya | Medium.
- [3] Jeff Hawkins, "On Intelligence: How a New Understanding of the Brain Will Lead to the Creation of Truly Intelligent Machines", 2004
- [4] PPT - Hierarchical Temporal Memory PowerPoint Presentation, free download - ID:299198 (slideserve.com)
- [5] Subutai Ahmad, "Hierarchical Temporal Memory" - Scholarpedia, 2012
- [6] Jeff Hawkins and Dileep George, "Hierarchical Temporal Memory", 2009

- [7] Subutai Ahmad, "Real-Time Multimodal Integration in a Hierarchical Temporal Memory Network", 2015
- [8] Jeff Hawkins and Subutai Ahmad, "Why Neurons Have Thousands of Synapses, A Theory of Sequence Memory in Neocortex", 2016
- [9] JamesLikesBrains Breaking down how a single HTM Neuron works? - Community - HTM Forum (numenta.org)
- [10] Neocortical Layers (umn.edu)
- [11] <https://www.vectorstock.com/royalty-free-vector/neuron-anatomical-structure-medical-education-vector-48845108>
- [12] "Yuwei Cui, Subutai Ahmad, Jeff Hawkins" <https://www.biorxiv.org/content/10.1101/085035v1.full>
- [13] Each distal dendrite segment of a cell has synapse connections Download Scientific Diagram (researchgate.net)
- [14] K. Hole, "The HTM Learning Algorithm. In: Anti-fragile ICT Systems. Simula SpringerBriefs on Computing, vol 1," 2016. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-30070-2_11.
- [15] S. A. J. Hawkins, "Why neurons have thousands of synapses, a theory of sequence memory in neocortex," 2016. [Online]. Available: <https://www.frontiersin.org/articles/10.3389/fncir.2016.00023/full>.
- [16] Guy-Evans, "Neurons (Nerve Cells) Structure, Function & Types," 2023. [Online]. Available: <https://simplypsychology.org/neuron.html>.